Measuring PECI performance and reliability

Jason Allred (jason.a.allred@intel.com), Jason M Bills (jason.m.bills@intel.com), Balakesan P Thevar (balakesan.p.thevar@intel.com)

1 Introduction

Platform Environment Control Interface (PECI[1]) is a debug interface that allows a Baseboard Management Controller (BMC) to communicate to the CPU via a single wire serial interface at speeds up to 2Mbps. It uses a set of service commands to provide a remote method for reading model specific registers (MSR), read and write to PCI endpoints, collect telemetry, download crash logs, and access a toolbox of package configuration space (PCS) options. Many of the topics covered here are detailed in the Intel System Management Spec[2] and Sapphire Rapids External Design Specification[3].

PECI is not a new interface, but two areas had recently changed the playing field. In earlier products, PECI relied on pcode/microcode, this design had the ability to impact CPU performance and was susceptible to choke points. In 10nm, a new silicon solution called OOBMSM[4] IP was created as a "database engine" or Local Translation Module (LTM) to offload several PECI requests as well as enhance security restrictions.

The second change was to how the PECI interface is presented to the user. PECI was originally accessed via an Intelligent Platform Management Interface (IPMI) command, and internally proxied from BMC through PCH firmware to the CPU. Legacy implementations such as ipmitool and ipmi.py relied on the RMCP+ protocol. A known vulnerability* in the RMCP+ protocol and a lack of centralized PECI filtering resulted in the recommendation that the PECI proxy over IPMI feature be deprecated.

As the BMC development team moved away from IPMI, they focused on a Restful API initiative called Redfish[5]. Our team added a new OEM† redfish command called SendRawPECI to fill a gap in validation. With OOBMSM and SendRawPECI in place, we initiated a series of **experiments** or tests that stressed our platforms enough to explore the limits of the hardware and highlight meaningful features to improve the reliability and speed of PECI by a factor of nearly 100.

The goal of this paper to explain the validation approach to achieve high quality of PECI services by the XEON CPUs by validating/measuring the PECI commands coverage – all commands on all devices, hardening against OOBMSM by using invalid headers and payloads, measuring performance of the Out-of-band interface, explore protocol timing and retry conditions, and provide CPU workload impact analysis.

2 Experiments

To verify the performance and reliability of PECI, our plan was to aim for speed first, then throw everything at the interface to try to break things alongside the 'devious minds' initiative mindset.

PECI commands coverage

Early in the investigation process, we learned that our silicon validation team had already created a flexible "peci_framework" including a script that would scrape the LTM code to provide a list of every single MSR offset and every PCI endpoint expected to be available over the PECI interface. This gave us the coverage, and a common means and method for sending large amounts of relevant data over PECI.

Hardening

For Hardening & Corruption coverage we focused on the format of PECI packets sent to a destination. Traditional fuzz testing was considered but ruled out since sending truly random data down a carefully curated tree structure like PECI would result in several quickly rejected results. Instead, a series of valid commands were chosen, then a single bit was flipped in each request. Command and thread id remained consistent. Other fields like offset, data, read & write length were corrupted. We called this experiment 'poison', but no checksums were being validated here. This 'poison' method allowed us to test how deeper flows were able to handle random commands gracefully.

Improving speed

To begin optimizing throughput, we identified areas that could act as bottlenecks to performance. Near the host/user side, legacy ipmitool scripts relied on loading executable and authenticating credentials for every transaction. The Redfish API allowed us to keep an authenticated session open, which was much more efficient. The most significant improvement we made came in the form of a 'Bulk send API' where several PECI commands were chained into one restful API HTTPS POST and handed off to the BMC for processing. The BMC firmware then handled the sending of each command over the PECI serial wire interface and then returned the requested reads back to the user. Additional throughput improvements were made after using a faster out-of-band interface over PCIe called MCTP (Management Component Transport Protocol[6]).

---

* RMCP+ & RAKP Vulnerability http://fish2.com/ipmi/remote-pw-cracking.html

† SendRawPECI is an Opt-in OEM feature as the command exposes a raw CPU debug interface

When stressing the interface over an extended period was important [See Figure 5 Impact on CPU performance], a ‘fast loop’ option was added to bypass python intensive code. With fast loops, an initial validation phase is run to ensure all requests were being properly handled and subsequent runs relied on cached payloads assuming they would return the same values.

Stressing the system

After our tests were running quickly, we needed to increase the workload sent to the OOBMSM handler. With Access to the LTM spec, we already had a list of every valid PECI requests that could be sent. We experimented with several ways to ‘throw the kitchen sink’ at our target. Iterative send loops followed the socket, thread, & register hierarchy of the PECI protocol. Once the Bulk API was put in use, we ran into timeout limitations. These timeouts were not hard limitations, but rather depended on how long we were willing to reasonably wait for data to be returned by the BMC. From our experiments, we discovered that the BMC itself did have a hard limit of 500 seconds. We needed to send smaller payloads. A good compromise between payload size and time limitations was to chunk payloads into specific areas. For example, one test read a single register across multiple threads; another sent a payload with all register offsets to a single socket/thread at a time.

Trimming out Retries and Rejected requests

Improving performance and stress introduced several challenges. We learned that performance of PECI requests relied heavily on how each command request in the full payload is handled. Some PECI requests are considered invalid and are returned immediately by the OOBMSM handler, while others take more time and require multiple retries to be successful. The PECI protocol uses defined return codes to communicate these results back to the user. Rejected commands, such as specific thread locations fused out of a specific CPU SKU, return quickly from OOBMSM with little processing. On the other hand, sleeping threads led to exercise retry loops that significantly increase the time spent on the BMC. When we began reading every combination of every command, it exposed holes in accessibility. Finding and documenting these holes took the form of a ‘sweep’ test that produced a heat map described later in the Results section [See Figure 3 MSR Sweep Heatmap].

Since a series of valid PECI requests provided the most accurate measure of performance and stress, we needed to first validate each command and related return code to craft a solid payload that would not trigger retried or rejected conditions. We started with a list of known valid offsets and endpoints, and then scanned a known good MSR offset for valid CPU threads, tagging any threads that are awake as usable for stressing. If desired, c-states could be disabled as a workaround to prevent any threads from sleeping. These tests later allowed us to demonstrate the usefulness of a new feature called ‘core wake on PECI’ which reduced retry conditions and removed the need for the disable c-state workaround.

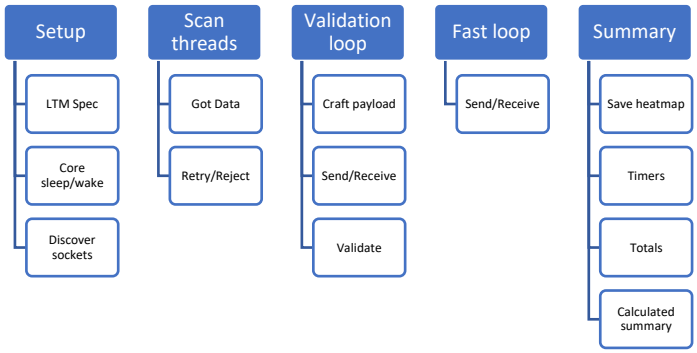

Figure 1 Test flow

While the bulk API for SendRawPECI improved performance, it caused a test design wrinkle. Sending multiple requests in a single payload meant that crafting requests and validating returned data could not be tightly coupled. This was solved by tracking what was sent and expected to be received, while deferring validation processing to software callbacks. Using validation callbacks also allowed us to handle retry conditions where only some of the requests were handled by the BMC before a timeout.

Measuring throughput

Once we started calculating performance metrics like commands per second and kbps, we wanted to make sure bottlenecks intrinsic to the test environment were not perturbing the results. Time spent in python and time over the network waiting for a response were easy to track with python’s perf_counter(), but time spent sending PECI traffic from the BMC to OOBMSM was more difficult. An elegant solution was to have the BMC return time used to process a request in the redfish API. We called this property ‘ElapsedTimeMs’ and it allowed us to quantify and optimize time spent stressing the internal PECI bus. [See Figure 2 MSR flood performance]

3 Results & Discussion

The purpose of this project was to measure performance and reliability of the new Redfish/OOBMSM PECI solution.

Three main results came from our experiments:

- Performance of MSR flood test
- Heatmap of MSR command & thread sweep
- Impact of CPU performance under PECI stress

Commands coverage and hardening

Fortunately, the OOBMSM proved an exceptionally reliable solution for rejecting unknown and handling known endpoints. We were not able to read unknown endpoints or crash the system using the hardening test. We did find two valid offsets where we were able to remotely crash the system with an MSR read, one of which has been fixed at the time of this paper.

MSR flood Performance metrics

The table below shows the results of our first experiment where throughput and other statistics are measured. Here we measure MSR read performance and compare combinations of sequential, bulk, serial/wire, MCTP and fast loop options. As noted in the profiling challenge discussion, time spent in Python became a larger factor after performance improvements were in place. Because of this, BMC time was used in commands per second and kbps calculations when available. We can also see that network time measured by an inner perf_counter loop was comparable to time reported by the BMC.

Test	sent	passed	failed	skipped	Python time	Network time	BMC time	cmd per second	tx kbps	rx kbps
single wire	6885	6610	275	320	2.0221	120.13	n/a	57.3149	0.2876	0.5171
single wire*	6669	1322	55	5612	3.0137	118.42	n/a	56.3175	0.2917	0.5246
bulk wire	6885	6610	275	320	11.5240	3.60	3.211	1911.2126	10.7580	19.3463
bulk wire*	6669	1322	55	5356	4.0139	3.66	3.26	1821.6942	10.7423	19.2469
bulk mctp	6885	6610	275	320	11.5644	1.29	0.846	5356.0022	40.8322	73.4291
bulk mctp*	6669	1322	55	5356	3.8286	1.29	0.84	5167.3684	41.6905	74.6964

Figure 2 MSR flood performance

*fast_loop enabled - Fast loops pushed down the time spent in Python but did not significantly change throughput as performance calculations were based only on BMC time or Network time.

MSR thread sweep Heatmap

When MSRs (Model Specific Registers) are read, the PECI API requires a register offset and a thread id to specify which thread to read from. We started with the assumption that we could read all registers with all threads. The experiment called ‘sweep’ highlighted how that assumption changed. In [Figure 3 MSR Sweep Heatmap] the rows record register name with address read, while columns show thread id’s that were used to in the PECI RdIaMSR sweep test. Values are PECI return codes, 0x40 indicates success, 0x90 (marked red) indicates the resource was unavailable. The heatmap allowed us to have productive discussions with system architecture experts and quickly zero in on expected & unexpected behavior.

	MSR	Thread 0	Thread 1	Thread 2	Thread 3	Thread 4	Thread 5	Thread 6	Thread 7	Thread 8	Thread 9	Thread 10	Thread 11	Thread 12	Thread 13	Thread 14	Thread 15	Thread 16	Thread 17	Thread 18	Thread 19
1	rdIaMsr 0x10 CORE_MSR	0x90	0x90	0x40	0x40	0x40	0x40	0x40	0x40	0x90	0x90	0x90	0x90	0x40	0x40	0x90	0x90	0x40	0x40	0x90	0x90
70	rdIaMsr 0x71b CORE_MSR	0x90	0x90	0x40	0x40	0x40	0x40	0x40	0x40	0x90	0x90	0x90	0x90	0x40	0x40	0x90	0x90	0x40	0x40	0x90	0x90
71	rdIaMsr 0x771 CORE_MSR	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
75	rdIaMsr 0x777 CORE_MSR	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
76	rdIaMsr 0xc90 CORE_MSR	0x90	0x90	0x40	0x40	0x40	0x40	0x40	0x40	0x90	0x90	0x90	0x90	0x40	0x40	0x90	0x90	0x40	0x40	0x90	0x90
107	rdIaMsr 0xd5e CORE_MSR	0x90	0x90	0x40	0x40	0x40	0x40	0x40	0x40	0x90	0x90	0x90	0x90	0x40	0x40	0x90	0x90	0x40	0x40	0x90	0x90
108	rdIaMsr 0x17 CR0_Chassis_Punit_CR	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
145	rdIaMsr 0x43f MCDDR_MSR_0	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
146	rdIaMsr 0x291 MCDDR_MSR_1	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
160	rdIaMsr 0x44f MCDDR_MSR_1	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
161	rdIaMsr 0x295 MCDDR_MSR_2	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
175	rdIaMsr 0x45f MCDDR_MSR_2	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
176	rdIaMsr 0x299 MCDDR_MSR_3	0x90	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
190	rdIaMsr 0x46f MCDDR_MSR_3	0x90	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
191	rdIaMsr 0x8210 MMAP_MS2IOSF_SCF_IO	0x40	0x40	0x90	0x40	0x40	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
217	rdIaMsr 0x82ff MMAP_MS2IOSF_SCF_IO	0x40	0x40	0x90	0x40	0x40	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
218	rdIaMsr 0x286 NCEVENTS_SCF_IOUBOX	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
298	rdIaMsr 0x433 SCF_MEM_MSR_0	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
299	rdIaMsr 0x290 SCF_MEM_MSR_1	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
303	rdIaMsr 0x443 SCF_MEM_MSR_1	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
304	rdIaMsr 0x294 SCF_MEM_MSR_2	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
308	rdIaMsr 0x453 SCF_MEM_MSR_2	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
309	rdIaMsr 0x298 SCF_MEM_MSR_3	0x90	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
315	rdIaMsr 0x463 SCF_MEM_MSR_3	0x90	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
314	rdIaMsr 0x285 SCF_UPI_MSR_0	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
318	rdIaMsr 0x417 SCF_UPI_MSR_0	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
319	rdIaMsr 0x287 SCF_UPI_MSR_1	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
323	rdIaMsr 0x41f SCF_UPI_MSR_1	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
324	rdIaMsr 0x288 SCF_UPI_MSR_2	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
328	rdIaMsr 0x423 SCF_UPI_MSR_2	0x90	0x90	0x40	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90	0x90
329	rdIaMsr 0x8000 UNICAST_GROUP1_CHA	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40
360	rdIaMsr 0x8301 UNICAST_GROUP1_CHA	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40	0x40

Figure 3 MSR Sweep Heatmap

Impact on CPU Performance

[Figure 4 PECI performance vs CPU load] shows the impact of CPU stress on PECI performance, with Stress-ng runs ranging from 0-100% CPU utilization. From this data, we can see that PECI performance is impacted. This is expected and good since we do not want PECI performance to fight CPU performance when resources are tight.

In [Figure 5 Impact on CPU performance] we compared SPEC-INT benchmark times under different conditions. This shows how increasing PECI payload sizes ranging from 0 to 1500 commands had no effect on benchmark run time.

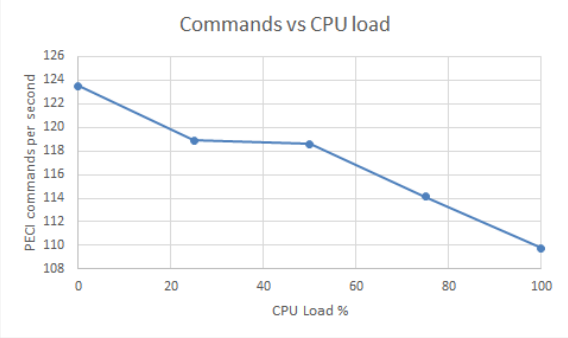

Figure 4 PECI performance vs CPU load

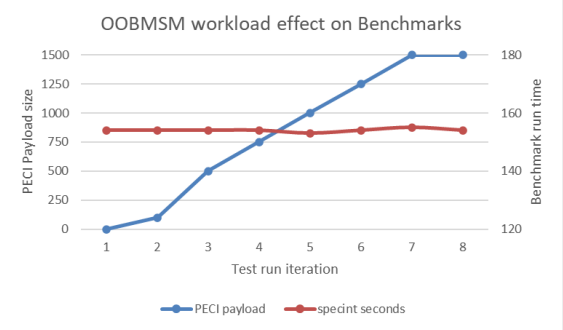

Figure 5 Impact on CPU performance

4 Conclusion and Future Plans / Use

Our experiments proved that PECI over OOBMSM is robust and capable. We saw that flooding will not perturb CPU performance and CPU stress can minimally impact PECI performance. The experiments stimulated several new platform level features related to performance including Bulk API, allowing partial data returned on timeout conditions, API knobs to tweak retry conditions, throughput improvements over MCTP, and a CCB to enable Core Wake on PECI read.

For further experiments, results would be more complete if we compared performance with legacy IPMI platforms. However, some major architectural differences blocked us from comparing them at this time. Stress tests using 'Write' were not a priority since PECI reads dominated the use case scenarios and WrIAMSR is not supported by the CPU. Further experiments using PCI & PCS writes are planned. We will also continue to monitor the customer reported bugs (AKA – Escaped bugs) and provide the necessary enhancements so that the escaped bugs will be zero or close to zero.

All experiments were converted into tests which are now used in BKC validation. Checkout style test was added to validate full coverage of each PECI command type, every endpoint access, all commands to all devices, and MSR reads to all cores. BKC testing has already caught high profile bugs such as machine checks on specific reads leading to updates. Beyond BKC, we plan to engage pcode/ucode patch team to add the MSR sweep as an integration test.

PECI has already proven itself as a useful debug and telemetry tool in datacenter environments. In the future, SendRawPECI and OOBMSM could expand to be used as a read-only alternative to at scale debug (ASD[7]) and real time telemetry.

5 Acknowledgments

Thanks to Chris Weldon for sponsoring the development work and encouraging us to write a paper, Charles T Scott for creating the PECI framework and to Jeff Gindlesperger for expanding on it, Boris Z Chernis for drafting up the initial PV tests. Thanks to Terry S Duncan for listening to my concerns about the future of PECI over IPMI and connecting me with BMC development. Special thanks to Dr. Jongsoon Park for peer review, and Bala for holding this all together and keeping us focused.

6 References

[1] PECI https://en-academic.com/dic.nsf/enwiki/4608863
[2] System Management Spec - https://www.intel.com/content/www/us/en/secure/design/internal/content-details.html?DocID=648590
[3] EDS Sapphire Rapids Server Processor External Design Specification (EDS), Volume One: Architecture
[4] OOBMSM & LTM https://wiki.ith.intel.com/display/ITSOOBMSM/Architecture
[5] Redfish https://www.dmtf.org/standards/redfish
[6] MCTP https://www.dmtf.org/sites/default/files/standards/documents/DSP2016.pdf
[7] ASD https://www.asset-intertech.com/resources/blog/2017/02/at-scale-debug/